PROJECT REPORT

PERSONAL FINANCE ADVISOR BOT

An AI-Powered Web Application for Budgeting, Savings Planning and Spending Analysis

Student Name	Swaraj Ombale
Branch	Artificial Intelligence and Data Science (AIDS)
Project Domain	AI Specialist – BFSI (Banking, Financial Services and Insurance)
Technology	Python, Flask, SQLite, SQLAlchemy, Google Gemini API
Source Code	github.com/swarajombale704-debug/personal-finance-advisor-bot

Abstract

Personal Finance Advisor Bot (presented in the application as **Finance AI**) is a web-based application that helps individuals understand their monthly finances and plan their savings. A registered user enters a monthly income, the monthly spending in four categories (Food, Transport, Dining and Entertainment) and a financial goal. The application stores this record, builds a structured request for the **Google Gemini API** and receives a recommended budget, a category-wise spending analysis and a set of saving suggestions in JSON format. If the Gemini service cannot be reached or returns an unusable response, the application falls back to a rule-based local analysis so that the user still receives a result.

The system is implemented in **Python** using the **Flask** framework, with **SQLite** accessed through the **SQLAlchemy** ORM. The user interface is built with HTML, CSS, JavaScript and Jinja2 templates. Authentication uses **Flask-Login** sessions and hashed passwords, and the application can be shared for demonstration through **Ngrok**. The interface presents a dashboard, an AI budget plan, a spending analysis, savings progress, a monthly summary, an analysis history and a print-based option to save the report as a PDF.

This report describes the problem, objectives, architecture, modules, workflow, implementation and testing of the project, and presents screenshots of the running application. The analysis produced by the application is advisory and informational; it is not professional financial advice and its accuracy is not guaranteed.

Keywords: personal finance, budgeting, Flask, Google Gemini API, SQLAlchemy, BFSI, artificial intelligence, expense analysis.

Table of Contents

Abstract i
1. Introduction 1
1.1 Background of Personal Finance Management 1
1.2 Need for an AI-Powered Finance Advisor 1
1.3 Overview of the Proposed System 1
2. Problem Statement 2
3. Objectives 3
4. Scope of the Project 4
4.1 Included in the Scope 4
4.2 Outside the Scope 4
5. Technology Stack 5
5.1 Explanation of the Main Technologies 5
6. System Architecture 7
6.1 Explanation of the Architecture Diagram 7
7. System Modules 8
7.1 User Registration and Login 8
7.2 Dashboard 8
7.3 Income and Expense Tracking 9
7.4 AI Financial Analysis 9
7.5 Budget Planning 9
7.6 Savings Goals 9
7.7 Monthly Summary 9
7.8 Analysis History 9
7.9 Report Printing / PDF 9
8. System Workflow 10
9. Implementation Details 12
9.1 Project Structure 12
9.2 Backend (Flask) 12
9.3 Database (SQLite and SQLAlchemy) 13
9.4 Authentication 13
9.5 Frontend 14
9.6 AI Integration with Google Gemini 14
9.7 Configuration and Security 15
9.8 Running the Application and Demonstration with Ngrok 15
10. Screenshots and Outputs 16
10.1 Login Page 16
10.2 Dashboard 17
10.3 AI Financial Analysis Results 17
10.4 Savings Progress and Monthly Summary 18
10.5 Analysis History
11. Testing and Validation 20
11.1 Approach 20
11.2 Test Cases 20
12. Results and Discussion 22
12.1 Functionality Demonstrated 22
12.2 Discussion 22
13. Limitations 23
14. Future Enhancements 24
15. Conclusion 25
References 26

List of Figures

Figure 1: Architecture of the Personal Finance Advisor Bot 7
Figure 2: Workflow of the Personal Finance Advisor Bot 10
Figure 3: Finance AI login page 16
Figure 4: Personal Finance Advisor dashboard with the AI Financial Analysis form 17
Figure 5: AI Financial Analysis results: AI Budget Plan and Spending Analysis 17
Figure 6: Savings Progress and Monthly Report 18
Figure 7: Analysis History 18
Figure 8: Registration page (to be inserted) 19
Figure 9: Saving Suggestions and Financial Overview (to be inserted) 19
Figure 10: Browser print dialog with Save as PDF (to be inserted) 19

List of Tables

Table 1: Common difficulties in personal finance management and the corresponding system support 2
Table 2: Technologies used and their roles 5
Table 3: Summary of system modules 8
Table 4: Step-by-step workflow 10
Table 5: Project files and their purpose 12
Table 6: Application routes 12
Table 7: Structure of the User table 13
Table 8: Structure of the FinancialRecord table 13
Table 9: JSON structure requested from the Gemini API 14
Table 10: Test cases and their status 20
Table 11: Sample analysis for a monthly income of ₹50,000 (Figures 5–7) 22
Table 12: Proposed future enhancements 24

1. Introduction

1.1 Background of Personal Finance Management

Personal finance management is the practice of planning, monitoring and controlling an individual's income, spending, savings and financial goals. Good money habits help a person meet everyday needs, prepare for emergencies and work steadily towards longer-term objectives such as building an emergency fund or saving for a major purchase. The basic activities involved are recording income, tracking expenses by category, comparing spending with a plan, and setting aside a portion of income as savings.

Digital banking, mobile payments and online shopping have made it easier to spend money and, at the same time, harder to keep a clear picture of where the money goes. Many people record their finances informally in notebooks or spreadsheets, or do not record them at all. In the Banking, Financial Services and Insurance (BFSI) domain, there is a growing interest in software that gives ordinary users clear, personalised guidance instead of leaving them to interpret raw account statements on their own.

1.2 Need for an AI-Powered Finance Advisor

A conventional expense tracker records numbers but leaves the interpretation to the user. A user who sees that a certain amount was spent on dining may not know whether that amount is reasonable for their income, which category deserves attention first, or how much they could realistically save each month. Human financial advisors can answer such questions, but they are not always accessible or affordable for everyday budgeting.

Large language models can read structured financial information and produce explanations in plain language. By combining such a model with a simple data-entry interface, a web application can offer instant, personalised feedback: a suggested budget, a comment on every spending category and practical saving ideas that take the user's stated goal into account. The Personal Finance Advisor Bot applies this idea using the Google Gemini API, backed by a local rule-based fallback so that the application remains useful when the AI service is unavailable.

1.3 Overview of the Proposed System

The proposed system is a Flask web application with user accounts. After logging in, the user submits a monthly income, expense amounts for four categories and a financial goal. The server validates and stores the data, requests an analysis from Google Gemini, and returns the result to the browser, where it is displayed as a budget plan, a spending analysis, saving suggestions, a savings progress indicator and a monthly summary. Every submission is saved so that the user can review earlier analyses in the history section.

2. Problem Statement

Individuals frequently find it difficult to manage money consistently. Income arrives at intervals, expenses occur in many small amounts across different categories, and there is rarely a clear, up-to-date view of how much remains available to save. The main difficulties addressed by this project are summarised in Table 1, together with the way the system responds to each of them.

Table 1: Common difficulties in personal finance management and the corresponding system support

Difficulty	Effect on the user	Support provided by the system
Tracking income and expenses manually	Records are incomplete or abandoned; the monthly picture is unclear.	A single form captures monthly income and category expenses; totals and savings are calculated automatically.
Understanding spending patterns	Users cannot tell which category is too high compared with income.	Each category is shown with its amount, its percentage of income and an On Track or Overspending status.
Planning a realistic budget	Budgets are guessed and often unrealistic.	An AI-generated budget plan recommends an allocation for every category.
Setting and following savings goals	Goals remain vague and progress is not visible.	The user states a goal; a saving target and a savings progress indicator are displayed.
Receiving personalised advice	Generic tips do not reflect the user's own figures.	Gemini receives the user's income, expenses and goal and returns tailored saving suggestions.
Reviewing past months	Earlier figures are forgotten or lost.	Every analysis is stored per user and listed in the Analysis History.

3. Objectives

The main objectives of the Personal Finance Advisor Bot are listed below.

1. **Track income and expenses:** allow a registered user to enter monthly income and category-wise expenses and calculate total expenses and actual savings.
2. **Analyse spending patterns:** express each expense category as a percentage of income and classify it as On Track or Overspending with an explanatory message.
3. **Generate AI-based financial suggestions:** use the Google Gemini API to produce a personalised budget plan and practical saving suggestions.
4. **Help users plan budgets and savings goals:** record a financial goal, suggest a monthly saving target and show progress towards that target.
5. **Display financial summaries and history:** present a dashboard, a monthly summary and a history of all earlier analyses for the logged-in user.

The supporting objectives are to protect user data through authentication and password hashing, to keep the application usable when the AI service is unavailable through a local fallback analysis, and to provide a clear, responsive and visually consistent user interface.

4. Scope of the Project

4.1 Included in the Scope

- User registration, login and logout with hashed passwords and session management.
- Entry of monthly income, four expense categories (Food, Transport, Dining, Entertainment) and a free-text financial goal.
- AI-generated budget plan, spending analysis and saving suggestions through the Google Gemini API.
- A local rule-based fallback analysis used when the AI response is unavailable or invalid.
- Dashboard indicators, savings progress, monthly summary and a per-user history of analyses.
- Saving the displayed report as a PDF through the print function of the web browser.
- Local execution on a Flask server with optional public demonstration through Ngrok.

4.2 Outside the Scope

- Connection to bank accounts, payment apps or automatic import of transactions.
- Investment, tax, loan or insurance advice, and any claim of guaranteed financial accuracy.
- Recording of individual transactions; the system works with monthly totals per category.
- Multiple currencies; the present interface and prompts use Indian Rupees (₹).
- Production-grade deployment, mobile applications and multi-user administration features.

> **Note:** The advice generated by the application is informational. Users should not treat it as a substitute for a qualified financial professional.

5. Technology Stack

The project is built entirely with open-source or freely available technologies. Table 2 lists each technology together with its role in the application.

Table 2: Technologies used and their roles

Technology	Category	Role in the project
Python	Language	Implements the server-side logic, data models, validation, prompt construction and response handling.
Flask	Web framework	Defines the routes (login, analysis, history), handles requests and responses and renders templates.
SQLite	Database	Stores user accounts and financial records in a single local file (finance.db).
SQLAlchemy (Flask-SQLAlchemy)	ORM	Maps the User and FinancialRecord classes to database tables and performs queries without raw SQL.
HTML	Frontend	Provides the structure of the login, registration, dashboard and history pages.
CSS	Frontend	Provides the dark, card-based layout, sidebar navigation, colours and responsive grid.
JavaScript	Frontend	Validates the form, sends the analysis request with fetch, renders results and summaries and triggers printing.
Jinja2	Template engine	Used by Flask (render_template) to serve the HTML templates.
Flask-Login	Authentication	Manages user sessions, the login_required protection of routes and the current_user object.
Werkzeug security	Authentication	Creates and verifies salted password hashes with generate_password_hash and check_password_hash.
python-dotenv	Configuration	Loads the secret key and the Gemini API key from a local .env file kept outside version control.
Google Gemini API (google-genai)	Artificial intelligence	Receives the structured prompt and returns the budget plan, spending analysis and saving suggestions in JSON.
Ngrok	Demo / deployment	Creates a temporary public URL that tunnels to the local Flask server so that the application can be demonstrated remotely.

5.1 Explanation of the Main Technologies

Python and Flask form the core of the system. Flask is a lightweight framework that keeps the application small and readable: each feature is a Python function bound to a URL route. This suits a project of this size and keeps the source code easy to explain and extend.

SQLite with SQLAlchemy provides persistence without requiring a separate database server. The ORM allows the data model to be written as Python classes, and relationships (one user with many financial records) are expressed directly in code.

HTML, CSS, JavaScript and Jinja2 make up the frontend. The pages are served by Flask through Jinja2 templates, while JavaScript communicates with the backend using asynchronous requests and updates the page without reloading it.

Flask-Login protects every private page and API route, and password hashing ensures that plain-text passwords are never stored. **Google Gemini** supplies the intelligence of the system, while **Ngrok** makes it possible to show the locally running application to others during a demonstration.

6. System Architecture

The application follows a client–server architecture with a clear separation between the presentation layer (browser), the application layer (Flask) and the data and intelligence services (SQLite and Google Gemini). Figure 1 shows how a request travels through the system and how the analysis result returns to the user.

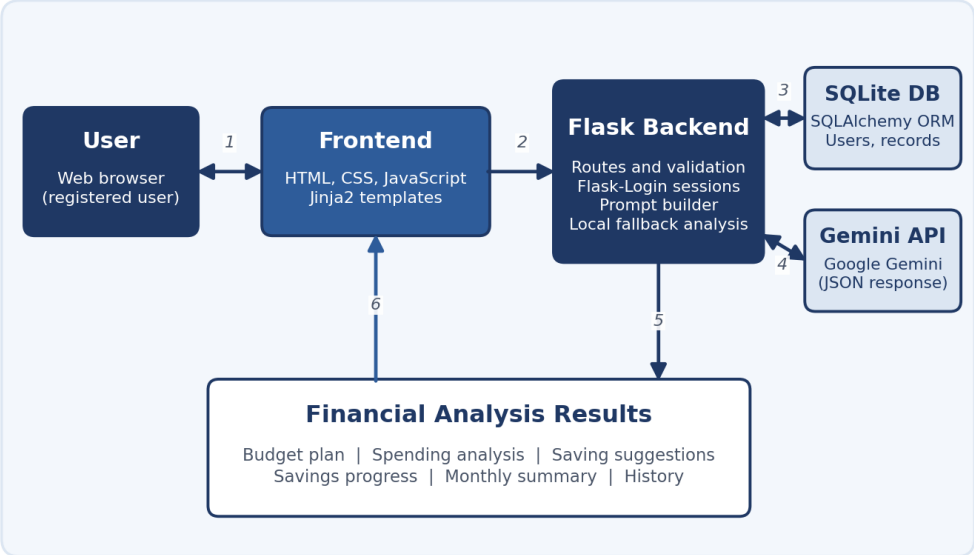


Figure 1: Architecture of the Personal Finance Advisor Bot

6.1 Explanation of the Architecture Diagram

1. **User and Frontend (1):** the user works in a browser. The frontend (HTML, CSS, JavaScript and Jinja2 templates) collects the input and sends requests to the server.
2. **Flask Backend (2):** the request reaches a Flask route. Flask-Login confirms that the user is logged in, and the backend validates the submitted income, expenses and goal.
3. **SQLite Database (3):** through SQLAlchemy, the backend reads user accounts during login and stores every submitted financial record so that it can appear later in the history.
4. **Gemini API (4):** the backend sends a structured prompt with the user's figures to Google Gemini and checks that the JSON reply contains the required keys. If the call fails or the reply is unusable, the local fallback analysis is used.
5. **Financial Analysis Results (5):** the backend returns the final JSON to the browser. It contains the budget plan, spending analysis, saving suggestions, actual savings and the source of the analysis.
6. **Display (6):** JavaScript in the frontend renders the results as cards, progress indicators and tables, and updates the dashboard, savings progress, monthly summary and history for the user.

7. System Modules

The application is organised into nine functional modules. Table 3 gives an overview, and each module is described in the sub-sections that follow.

Table 3**:** Summary of system modules

No.	Module	Purpose	Main implementation
1	User Registration and Login	Create accounts and authenticate users	/register, /login, /logout in app.py
2	Dashboard	Show income, expenses, savings, budget health and goal	templates/index.html
3	Income and Expense Tracking	Collect and store monthly income and category expenses	Analysis form and FinancialRecord model
4	AI Financial Analysis	Obtain budget plan, spending analysis and suggestions	/analyse route and Gemini client
5	Budget Planning	Display recommended allocation per category	AI Budget Plan card
6	Savings Goals	Record the goal, target and savings progress	Financial Goal and Savings Progress cards
7	Monthly Summary	Summarise income, expenses, savings and target	Monthly Report card
8	Analysis History	List earlier analyses of the logged-in user	/history and /history-page
9	Report Printing / PDF	Save the report as PDF through the browser	window.print() in saveReportPDF()

7.1 User Registration and Login

New users register with a full name, an email address and a password of at least six characters. The email address is converted to lower case and must be unique. The password is stored only as a salted hash. Existing users log in with their email address and password; on success Flask-Login creates a session and the user is taken to the dashboard. All other routes are protected, and an unauthenticated visitor is redirected to the login page.

7.2 Dashboard

The dashboard is the home screen. Three summary cards show Monthly Income, Total Expenses and Actual Savings. A Budget Health indicator shows how the figures relate to one another, and a Financial Goal card shows the current goal together with a suggested monthly target. A sidebar provides navigation to Dashboard, AI Analyse, Budget, Goals, Monthly Summary and History.

7.3 Income and Expense Tracking

The AI Financial Analysis form accepts the monthly income, the monthly amount spent on Food, Transport, Dining and Entertainment, and the financial goal. The server converts the values to numbers, rejects negative amounts and stores each submission as a financial record linked to the user. The system therefore tracks monthly category totals rather than individual transactions.

7.4 AI Financial Analysis

When the user selects Analyse My Finances, the browser sends the data to the /analyse route. The backend asks Google Gemini to prepare a personalised budget, to calculate the percentage of income spent in each category, to mark each category as On Track or Overspending, and to give three to five saving suggestions. A detailed financial overview is also displayed, including the savings rate, the largest expense, the income-to-expense ratio, spending alerts and a simple projection based on the current month's savings.

7.5 Budget Planning

The AI Budget Plan card lists the recommended allocation for each expense category, with the amount and the percentage of income. Fixed and variable expenses are separated in the data returned by the analysis, and a monthly saving target is proposed.

7.6 Savings Goals

The user enters a goal in plain text, for example an emergency fund or general saving. The Financial Goal card shows this goal, and the Savings Progress card compares the saving target with the current savings, showing the remaining amount and the percentage of the target achieved.

7.7 Monthly Summary

The Monthly Report presents four figures – Monthly Income, Total Expenses, Actual Savings and Saving Target – followed by a table with a status label for each line (Available, Spending, Saved and Target). It gives a quick overview of the month for the analysis that is currently displayed.

7.8 Analysis History

Each analysis is saved with its date and time. The history lists the records of the logged-in user, newest first, showing income, expenses, savings, the goal and the amount in each category. The View Analysis option reloads a selected record into the dashboard and the form.

7.9 Report Printing / PDF

The Save Report as PDF button calls the browser print function. In the print dialog, the user chooses Save as PDF as the destination to obtain a PDF copy of the displayed report. No server-side PDF generation is involved.

8. System Workflow

Figure 2 shows the complete flow of the application from the moment a user opens it until the analysis results are displayed, including the two error paths (failed login and invalid input) and the choice between the Gemini result and the local fallback.

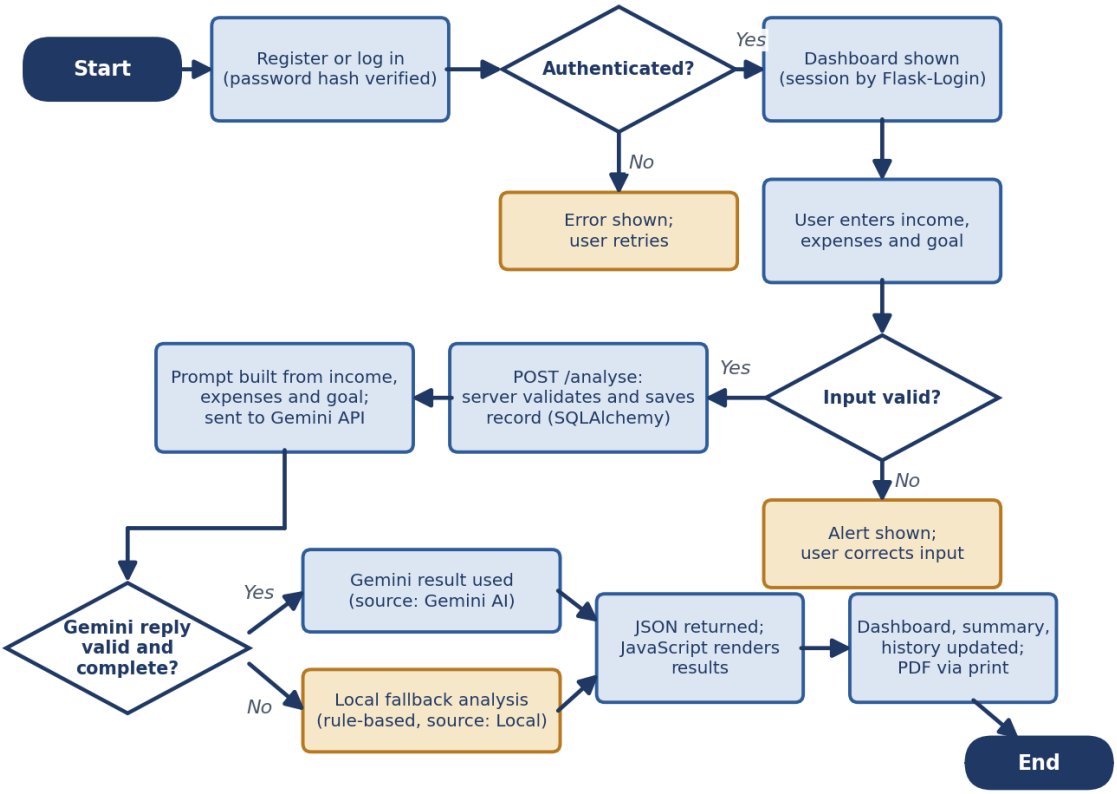


Figure 2: Workflow of the Personal Finance Advisor Bot

The steps are described in Table 4.

Table 4: Step-by-step workflow

Step	Actor	Action
1	User	Opens the application. An unauthenticated visitor is redirected to the login page.
2	User / Flask	Registers a new account or logs in. The submitted password is compared with the stored hash and a session is created for a valid user.
3	Flask	Displays the dashboard (index page) for the authenticated user.
4	User	Enters monthly income, expenses for the four categories and a financial goal, then selects Analyse My Finances.
5	JavaScript	Checks that the income is positive and the goal is not empty, then sends the data to /analyse as JSON.
6	Flask	Validates the data again (numeric values, no negative amounts, at least one expense above zero) and saves a financial record in SQLite.

Step	Actor	Action
7	Flask	Builds the prompt and calls the Gemini API. The reply is parsed as JSON and checked for the keys budget_plan, spending_analysis and saving_suggestions.
8	Flask	If the Gemini reply is valid, it is returned with the source Gemini AI. Otherwise the local fallback analysis is returned.
9	JavaScript	Renders the budget plan, spending analysis, saving suggestions and detailed overview, and updates the dashboard, savings progress and monthly summary.
10	User	Reviews the results, opens earlier analyses from the history, or saves the report as a PDF through the print dialog.

9. Implementation Details

9.1 Project Structure

The project is organised as a small Flask application. The main files are listed in Table 5.

Table 5: Project files and their purpose

File / folder	Description
app.py	Application entry point: configuration, database models, authentication routes, analysis route, history API, Gemini integration and fallback analysis.
templates/login.html	Login page.
templates/register.html	Registration page.
templates/index.html	Main page: dashboard, analysis form, results, savings progress, monthly summary and history section, with the page JavaScript.
templates/history.html	Stand-alone page that lists previous analyses.
static/css/style.css	Shared stylesheet.
requirements.txt	Python package list for installation.
.gitignore	Excludes the virtual environment, the .env file with secrets and the local database from version control.

9.2 Backend (Flask)

The backend is a single Flask module. Configuration values such as the secret key and the Gemini API key are read from environment variables, which are loaded from a local .env file. The routes are summarised in Table 6.

Table 6: Application routes

Route	Method	Login required	Purpose
/	GET	Yes	Dashboard page
/register	GET, POST	No	Show the registration form and create an account
/login	GET, POST	No	Show the login form and authenticate the user
/logout	GET	Yes	End the session and return to the login page
/analyse	POST	Yes	Validate input, save the record and return the analysis as JSON
/history	GET	Yes	Return the logged-in user's records as JSON, newest first
/history-page	GET	Yes	Show the history page

The /analyse route first checks that the request contains data, that the income is a number greater than zero and that the expenses are a non-empty set of non-negative numbers whose total is greater than zero. Only then is the record saved and the analysis requested.

9.3 Database (SQLite and SQLAlchemy)

Two tables are created automatically at start-up with db.create_all(). A user can have many financial records; deleting a user also deletes the related records (cascade delete). The expenses of each record are stored as a JSON text so that the category set can be extended without changing the table structure.

Table 7: Structure of the User table

Field	Type	Description
id	Integer, primary key	Unique identifier of the user
full_name	String(120), required	Name entered at registration
email	String(120), unique, required	Login identifier, stored in lower case
password_hash	String(255), required	Salted password hash (the password itself is never stored)
created_at	DateTime	Time of registration (UTC)

Table 8: Structure of the FinancialRecord table

Field	Type	Description
id	Integer, primary key	Unique identifier of the record
user_id	Integer, foreign key to user.id	Owner of the record
income	Float, required	Monthly income entered by the user
expenses	Text (JSON), required	Category-wise expenses, for example Food, Transport, Dining, Entertainment
goal	Text, optional	Financial goal entered by the user
created_at	DateTime	Time of the analysis (UTC)

9.4 Authentication

Registration validates the name, the email address and the password length, checks that the email is not already registered and stores only the password hash created by Werkzeug. At login, the entered password is verified with check_password_hash and, if valid, login_user() starts the session. The decorator @login_required protects every private route, and the login manager redirects visitors to the login page. The relevant statements are shown below.

Code 1: Password hashing and login (from app.py)

```
# Registration: store only the hash of the password
password_hash = generate_password_hash(password)
user = User(full_name=full_name, email=email,
            password_hash=password_hash)

# Login: verify the hash and start a session
user = User.query.filter_by(email=email).first()
if user and check_password_hash(user.password_hash, password):
    login_user(user)
    return redirect(url_for("home"))
```

9.5 Frontend

The frontend uses a dark theme with a fixed sidebar and a grid of cards. Fonts and icons are loaded from public content delivery networks. The main page contains the summary cards, the Budget Health indicator, the Financial Goal card, the analysis form and the areas where results are inserted. JavaScript performs three tasks: it validates the form and sends the request with the fetch function, it renders the returned JSON into the budget plan, the spending analysis, the saving suggestions and the detailed overview, and it updates the dashboard, savings progress and monthly summary. Text received from the server is escaped before it is inserted into the page. The history section requests /history and builds a list of records, each with a View Analysis button.

9.6 AI Integration with Google Gemini

The integration consists of a prompt builder, a call to the Gemini client, a JSON extractor and a validation step. The prompt presents the model as a personal finance advisor, supplies the income, the expenses and the goal, lists the tasks to be performed and requires that only valid JSON is returned in a fixed structure. Table 9 summarises the expected structure.

Table 9: JSON structure requested from the Gemini API

JSON key	Content
budget_plan	Fixed expenses, variable expenses (category, amount, percentage) and a monthly saving target.
spending_analysis	One entry per category with amount, percentage of income, status (On Track or Overspending) and an explanatory message.
saving_suggestions	Three to five suggestions, each with an estimated saving amount and the target it supports.

Language models may wrap JSON in additional text or Markdown fences. For this reason the function extract_json() first tries to parse the whole reply, then looks for a fenced block and finally for the text between the first and the last brace. The result is accepted only if all three required keys are present; the application then adds the actual savings and marks the source as Gemini AI. In every other case the function create_fallback_analysis() is used. It calculates the percentage for each category, marks a category as Overspending when it is above 15 percent

of income, proposes a saving target of 20 percent of income and returns a fixed set of general saving suggestions. The relevant logic is shown below in condensed form.

Code 2: Gemini call with validation and fallback (condensed from app.py)

```
response = client.models.generate_content(
    model=MODEL_NAME, contents=prompt)
result = extract_json(response.text)

required = ("budget_plan", "spending_analysis",
            "saving_suggestions")
if result and all(k in result for k in required):
    result["actual_savings"] = income - total_expenses
    result["analysis_source"] = "Gemini AI"
    return jsonify(result)

# any error or invalid reply: use the local rule-based analysis
return jsonify(fallback_result)
```

9.7 Configuration and Security

- The Gemini API key and the Flask secret key are read from environment variables and are not written in the source code. The .env file and the database file are listed in .gitignore.
- Passwords are stored only as salted hashes, and private routes require an authenticated session.
- Every database query on records is filtered by the identifier of the logged-in user, so users can see only their own history.
- If no API key is configured, the application still runs and uses the local fallback analysis.

9.8 Running the Application and Demonstration with Ngrok

The application runs on the Flask development server at http://127.0.0.1:5000. For a remote demonstration, Ngrok creates a temporary public address that forwards requests to this local port.

Code 3: Starting the application and the Ngrok tunnel

```
# Terminal 1: start the Flask application
python app.py

# Terminal 2: expose local port 5000 through Ngrok
ngrok http 5000
```

10. Screenshots and Outputs

This chapter shows screenshots of the application running locally on http://127.0.0.1:5000. Only screens that were actually captured are shown. Where a screen is needed for completeness but was not provided, a clearly labelled placeholder is left so that the screenshot can be inserted later.

10.1 Login Page

Figure 3 shows the login page. The address bar shows that the dashboard address was redirected to /login?next=%2F, which is the expected behaviour for a visitor who is not logged in.

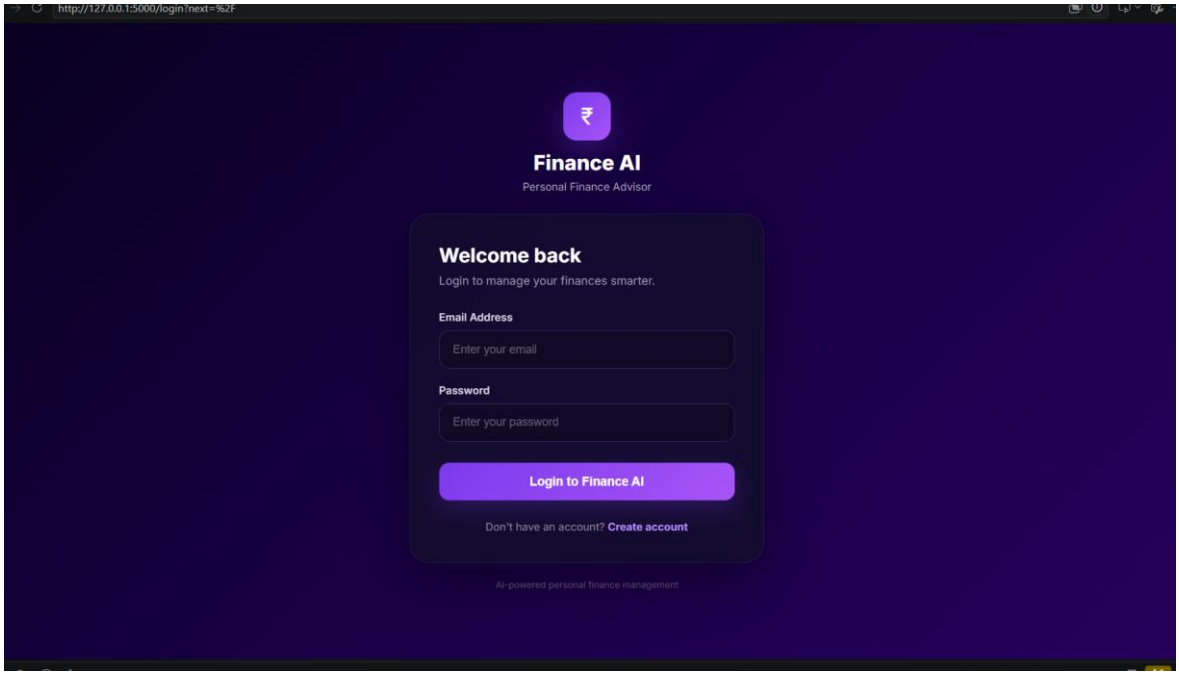


Figure 3: Finance AI login page

10.2 Dashboard

Figure 4 shows the dashboard before an analysis has been submitted: the summary cards, Budget Health and Financial Goal show zero values, and the AI Financial Analysis form is displayed below them. The grey figures inside the input boxes are example values.

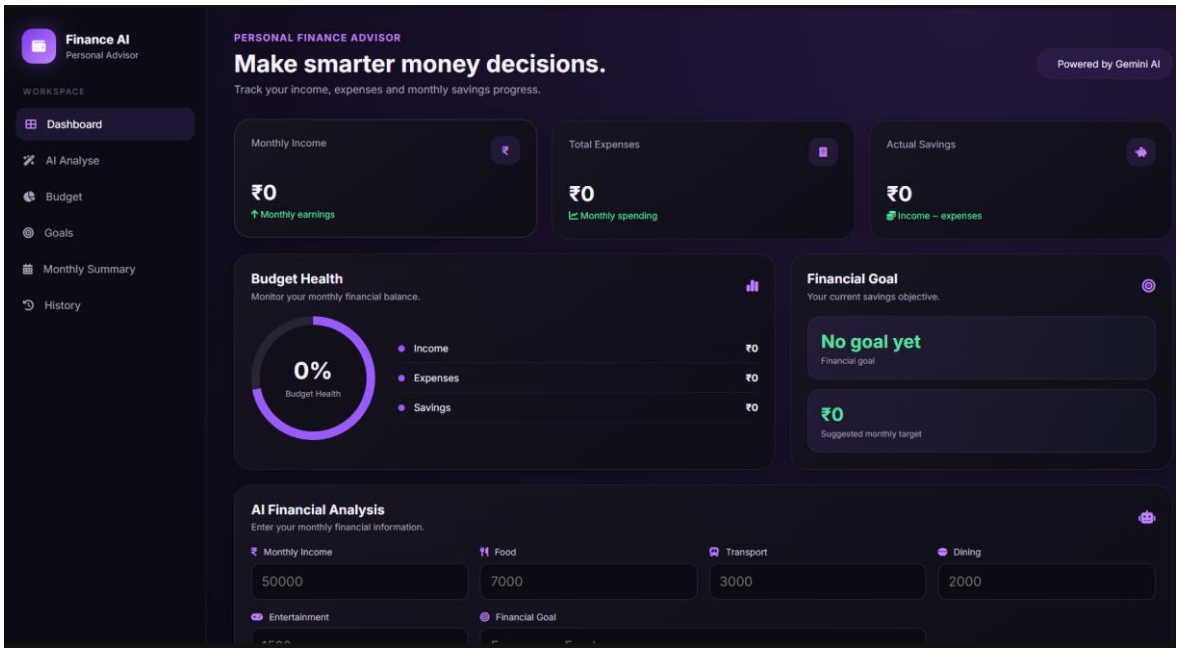


Figure 4: Personal Finance Advisor dashboard with the AI Financial Analysis form

10.3 AI Financial Analysis Results

Figure 5 shows the AI Budget Plan and the Spending Analysis for a monthly income of ₹50,000 with expenses of ₹2,000 on Food, ₹4,500 on Transport, ₹3,500 on Dining and ₹1,500 on Entertainment. Each category is shown with its amount, its percentage of income, a status and a short message. The Save Report as PDF button is visible above the budget plan.

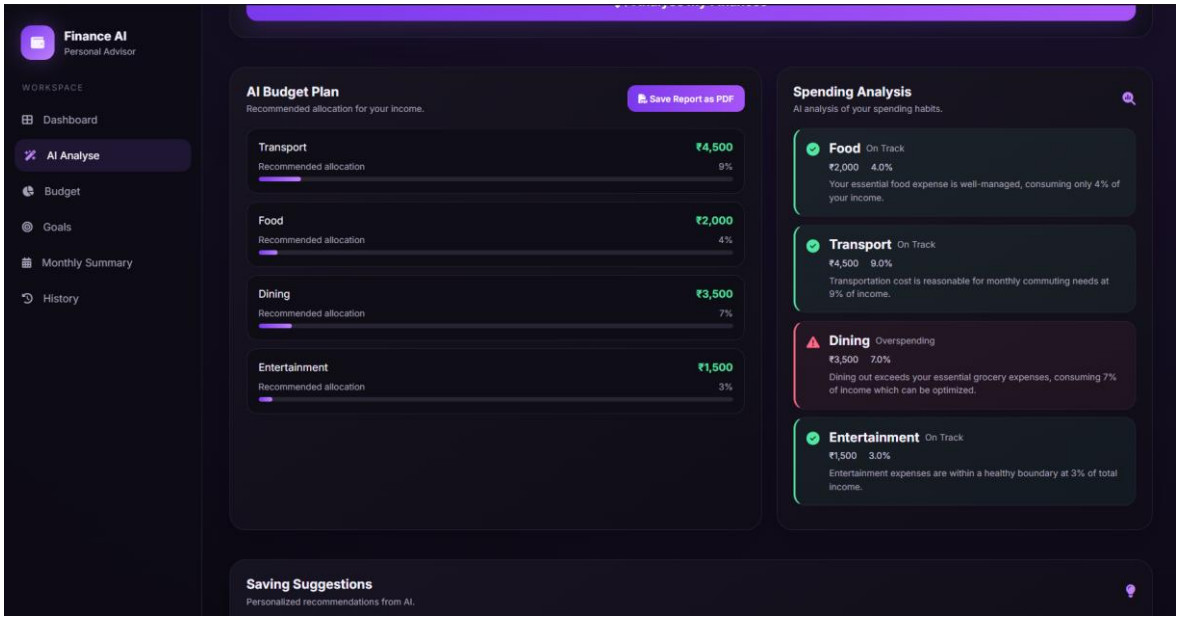


Figure 5: AI Financial Analysis results: AI Budget Plan and Spending Analysis

10.4 Savings Progress and Monthly Summary

Figure 6 shows the Savings Progress indicator and the Monthly Report. For the same analysis, the report shows income of ₹50,000, expenses of ₹11,500, actual savings of ₹38,500 and a saving target of ₹10,000.

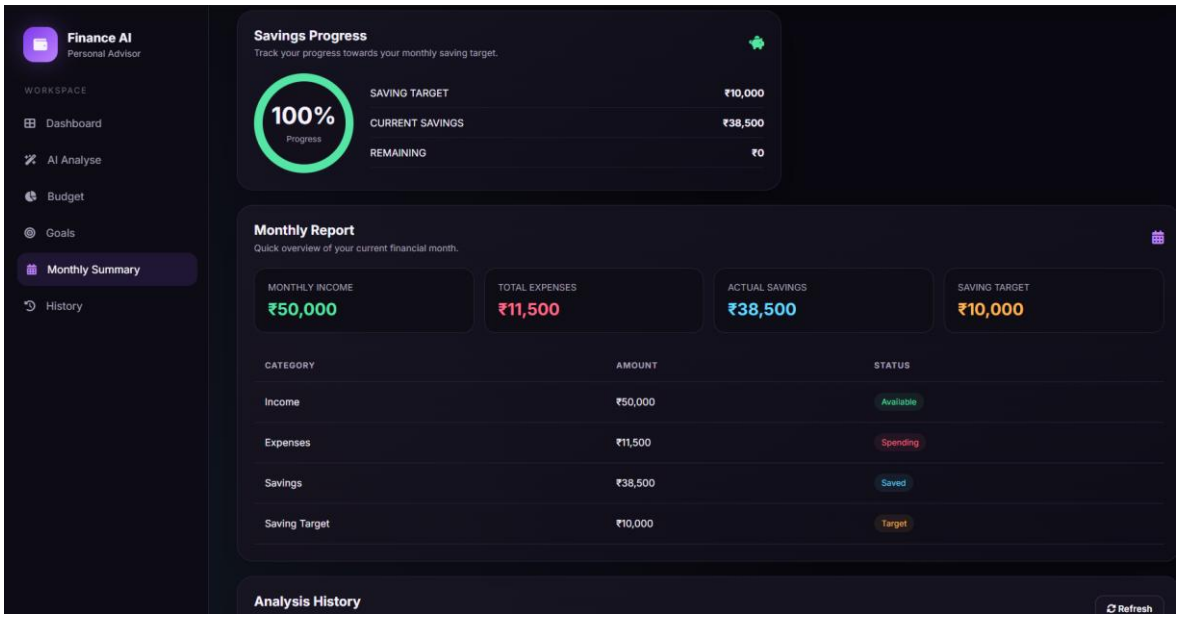


Figure 6: Savings Progress and Monthly Report

10.5 Analysis History

Figure 7 shows the Analysis History, which lists previous analyses of the user with their date and time, income, expenses, savings, goal and category amounts, each with a View Analysis button. The highlighted text in the first entry is a text selection in the browser and not a feature of the application.

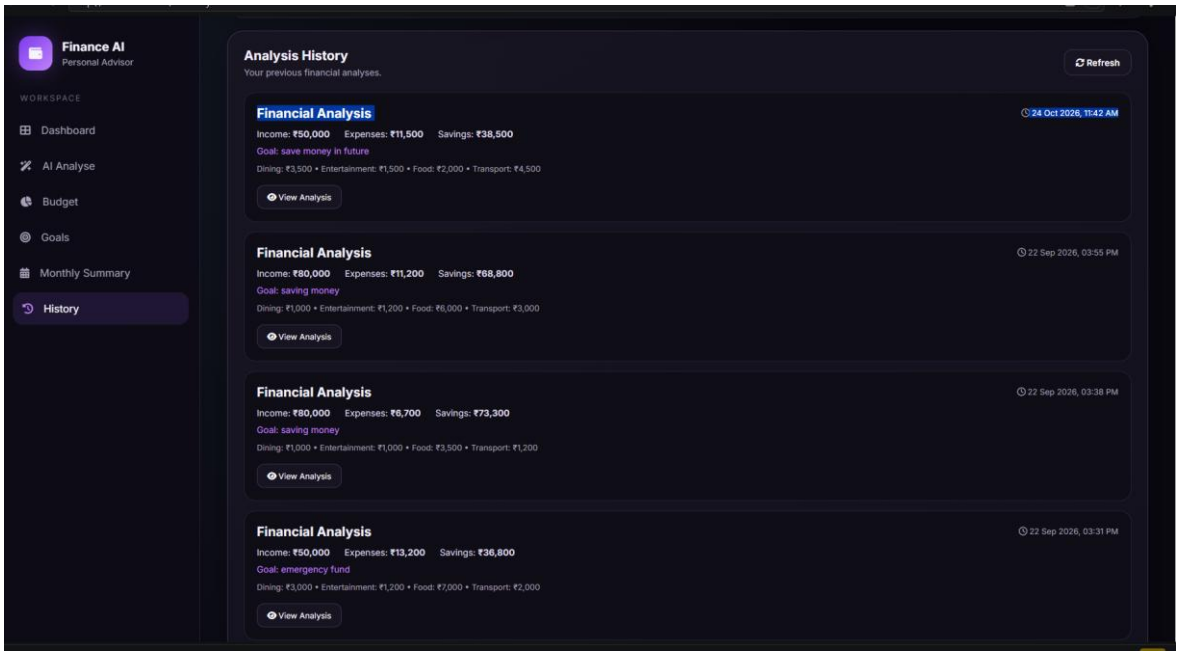


Figure 7: Analysis History

11. Testing and Validation

11.1 Approach

The application was checked through manual functional testing in a web browser. A test case is marked **Passed** only when the screenshots provided with this report show the expected behaviour. Test cases for which no evidence is available at the time of writing are marked **To Be Verified** and should be executed and updated by the student before final submission. The cases marked To Be Verified are expected from the implemented code, but they have not been demonstrated in this report.

11.2 Test Cases

Table 10**:** Test cases and their status

ID	Feature	Expected result	Actual result	Status
TC-01	Access without login	A visitor who is not logged in is redirected to the login page.	Login page opened at /login?next=%2F (Figure 3).	**Passed**
TC-02	Login page display	Email and password fields, login button and Create account link are shown.	Page displayed as expected (Figure 3).	**Passed**
TC-03	Login with valid credentials	The user reaches the dashboard.	Dashboard and later screens were captured in a logged-in session (Figures 4–7).	**Passed**
TC-04	Registration with valid details	Account is created and the user is sent to the login page with a success message.	Not yet recorded.	**To Be Verified**
TC-05	Login with wrong password	Message “Invalid email or password.” is shown.	Not yet recorded.	**To Be Verified**
TC-06	Registration with duplicate email or short password	An error message is shown and no account is created.	Not yet recorded.	**To Be Verified**
TC-07	Dashboard display	Summary cards, Budget Health, Financial Goal and analysis form are displayed.	Displayed as shown in Figure 4.	**Passed**
TC-08	Form validation	Zero income or an empty goal is rejected with a message.	Not yet recorded.	**To Be Verified**
TC-09	AI budget plan	A recommended allocation is shown for each expense category.	Allocations for Transport, Food, Dining and Entertainment shown (Figure 5).	**Passed**
TC-10	Spending analysis	Each category shows amount, percentage, status and message.	All four categories shown with status On Track or Overspending (Figure 5).	**Passed**

ID	Feature	Expected result	Actual result	Status
TC-11	Saving suggestions	Three to five saving suggestions are displayed.	Section heading visible only; contents not captured (Figure 5).	To Be Verified
TC-12	Savings calculation	Savings equal income minus total expenses.	₹50,000 − ₹11,500 = ₹38,500 shown (Figure 6).	Passed
TC-13	Savings progress	Target, current savings, remaining amount and percentage are shown.	₹10,000 target, ₹38,500 current, ₹0 remaining, 100% (Figure 6).	Passed
TC-14	Analysis history	Earlier analyses are listed with date, income, expenses, savings, goal and categories.	Four entries with timestamps visible (Figure 7).	Passed
TC-15	View Analysis from history	A selected record is reloaded into the dashboard and form.	Not yet recorded.	To Be Verified
TC-16	Fallback analysis	When Gemini is unavailable, a local analysis is returned.	Not yet recorded.	To Be Verified
TC-17	Save Report as PDF	The browser print dialog opens and the report can be saved as PDF.	Button visible (Figure 5); print output not captured.	To Be Verified
TC-18	Logout	The session ends and the login page is shown.	Not yet recorded.	To Be Verified
TC-19	Demonstration through Ngrok	The application opens through the public Ngrok address.	Not yet recorded.	To Be Verified

12. Results and Discussion

12.1 Functionality Demonstrated

The screenshots demonstrate that the application provides a protected login, a dashboard with an analysis form, an AI-generated budget plan and spending analysis, a savings progress indicator, a monthly summary and a history of earlier analyses. Table 11 repeats the figures from the sample analysis shown in Figures 5 to 7. The percentages are those displayed in the application.

Table 11**:** Sample analysis for a monthly income of ₹50,000 (Figures 5–7)

Category	Amount (₹)	Share of income (%)	Status shown
Food	2,000	4.0	On Track
Transport	4,500	9.0	On Track
Dining	3,500	7.0	Overspending
Entertainment	1,500	3.0	On Track
Total expenses	**11,500**	**23.0**	**–**

With an income of ₹50,000 and total expenses of ₹11,500, the application reports actual savings of ₹38,500. The user's saving target of ₹10,000 is therefore fully covered, and the Savings Progress indicator shows 100 percent with no remaining amount.

12.2 Discussion

The results show that the application turns a handful of inputs into an organised and readable report. The user does not need to calculate percentages or compare categories manually, because the application does this and explains each result in plain language. Keeping every analysis in the history lets the user compare different months.

The status of each category is a judgement made by the language model and is not a single fixed threshold. In the sample, Dining at 7.0 percent of income is marked Overspending, whereas Transport at 9.0 percent is marked On Track, and the accompanying messages explain the reasoning in each case. By contrast, the local fallback analysis uses a fixed limit of 15 percent of income per category. Users should therefore treat the classifications as guidance, review them against their own circumstances, and expect that the wording and the suggestions may differ between runs.

> **Disclaimer:** No performance measurements are reported, because none were recorded. The analysis is generated by an AI model and the accuracy or suitability of the advice is not guaranteed.

13. Limitations

- **Internet and API availability:** the AI-generated analysis requires an internet connection, a valid Gemini API key and an available service. When these are not available, the application uses the local rule-based analysis, which is simpler, applies fixed rules and gives general suggestions.
- **Variable AI output:** the content, tone and classification returned by a language model can vary, and the recommended allocations may closely follow the entered figures. The output is advisory and must be reviewed by the user.
- **Monthly category totals only:** the system accepts four fixed categories and monthly totals; it does not record individual transactions, custom categories or recurring items, and it does not connect to bank accounts.
- **Summary based on the displayed analysis:** the dashboard, savings progress and monthly summary reflect the analysis that is currently displayed or selected from the history, not an automatic aggregation of several records for a calendar month.
- **Time display:** record times are created with UTC and shown without conversion, so the history may show UTC rather than local time.
- **Currency:** only Indian Rupees (₹) are supported.
- **Deployment:** the project uses the Flask development server and a local SQLite file, and Ngrok provides a temporary public address. This arrangement is suitable for demonstration but not for many simultaneous users or production use.
- **Not professional advice:** the application does not replace a qualified financial advisor and does not consider taxes, investments, loans or insurance.

14. Future Enhancements

The following improvements are proposed for later versions of the project.

Table 12**:** Proposed future enhancements

Area	Proposed enhancement	Expected benefit
Advanced analytics	Charts for category shares and month-to-month trends; comparison of several months from the stored history.	Clearer view of long-term spending patterns.
Additional report formats	Server-side PDF generation and export to Excel or CSV.	Reports that can be shared or archived without using the print dialog.
Improved forecasting	Projection of savings and goal completion dates using the stored history and statistical or machine-learning models.	More realistic planning of savings goals.
Flexible tracking	Custom categories, individual transactions and recurring expenses.	More detailed and accurate input.
Enhanced security	Stronger password rules, CSRF protection, rate limiting, email verification, optional two-factor authentication, HTTPS and disabling debug mode in production.	Better protection of user accounts and data.
Maintainability and deployment	Complete dependency list in requirements.txt (including Flask-SQLAlchemy and Flask-Login), automated tests, a production web server and a production database.	Easier installation, reliable behaviour and scalability.
Localisation	Conversion of times to the user’s time zone and support for additional currencies and languages.	Wider usability.

15. Conclusion

The Personal Finance Advisor Bot demonstrates how a conventional web application can be extended with a generative AI service to support everyday financial decisions. Using Python, Flask, SQLite with SQLAlchemy, a Jinja2-based frontend, Flask-Login and the Google Gemini API, the project allows a user to record monthly income and expenses, obtain a budget plan, understand spending by category, follow progress towards a saving target and review earlier analyses.

Important design decisions are the protection of private data through authentication and password hashing, the use of a strict JSON structure for the AI response, and a local fallback analysis that keeps the application useful when the AI service is not available. The limitations of the present version – fixed categories, dependence on an external service, advisory-only output and demonstration-level deployment – are clearly identified, and a set of enhancements has been proposed. The project meets its stated objectives and provides a sound base for further development in the BFSI domain.

References

[1] Python Software Foundation, “Python Documentation,” https://docs.python.org/3/

[2] Pallets Projects, “Flask Documentation,” https://flask.palletsprojects.com/

[3] Pallets Projects, “Flask-SQLAlchemy Documentation,” https://flask-sqlalchemy.palletsprojects.com/

[4] SQLAlchemy Project, “SQLAlchemy Documentation,” https://docs.sqlalchemy.org/

[5] SQLite Consortium, “SQLite Documentation,” https://www.sqlite.org/docs.html

[6] Flask-Login Contributors, “Flask-Login Documentation,” https://flask-login.readthedocs.io/

[7] Pallets Projects, “Jinja Template Designer Documentation,” https://jinja.palletsprojects.com/

[8] Pallets Projects, “Werkzeug Documentation: Security Helpers,” https://werkzeug.palletsprojects.com/

[9] Google, “Gemini API Documentation,” https://ai.google.dev/gemini-api/docs

[10] ngrok, “ngrok Documentation,” https://ngrok.com/docs

[11] Mozilla, “MDN Web Docs: HTML, CSS and JavaScript,” https://developer.mozilla.org/

[12] S. Ombale, “Personal Finance Advisor Bot,” project source code, https://github.com/swarajombale704-debug/personal-finance-advisor-bot